

LAB 2 TRIALS AND ERRORS (AND SOLUTIONS)

BY VIC GRIGORYEV (VGRIGORY)

1. *Get a Minix System*

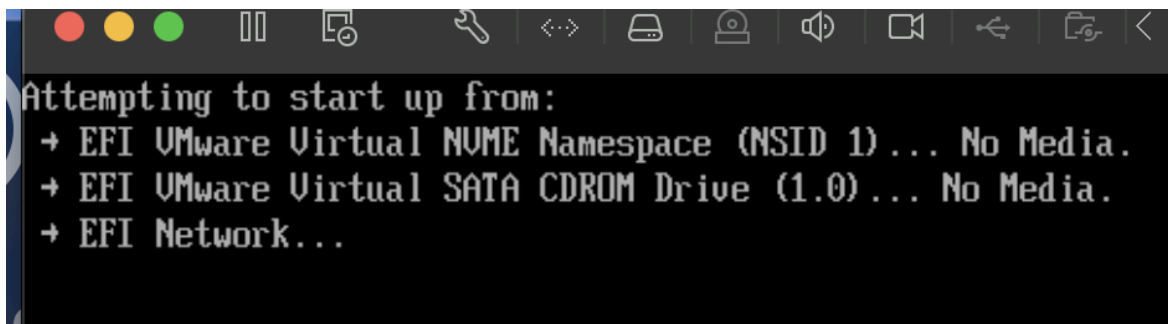
Approach:

Download, install, and run a way to create and boot a virtual (or native) Minix system. Because I have important files on my computers (and I'd rather keep my OS's and hard disks intact) I decided to go the virtual machine route.

Problems Encountered:

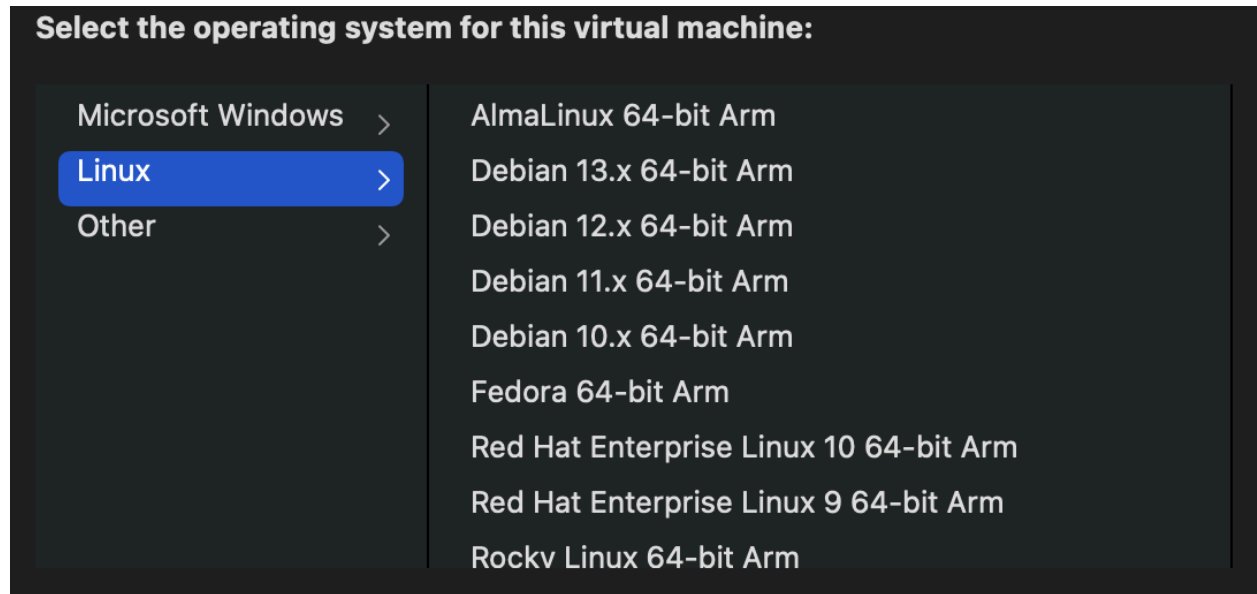
I work on an M1 Macbook Air at school since my 18 year old freshman self decided a Windows gaming laptop was a good thing to have for college. It weighs about 10 pounds and I keep it hooked up to my monitor at home and work on it for anything that doesn't involve code.

While I had my Mac out, I decided to go get VMWare Fusion since my dad used to work at VMWare and I had some experience screwing around on it on a previous work laptop when I was younger. Also bonus that it was now free! I tried to start an "other" Linux virtual machine and replace the VMDK Fusion generated with the given one and it refused to start. No matter what settings I changed, the result was the same: the system could not boot from anything and thus just waited for a network bootloader.



Looking for information online wasn't any help because there wasn't anything Fusion related - mostly just stuff about its Windows and Linux version, Workstation, and how to install Minix on there.

It was only when I decided to investigate why the only systems listed on the setup screen were x64 ARM did I realize what the issue was... Fusion was virtualizing the machines and not truly emulating them. Apple's limiting but powerful ecosystem strikes again!



That meant I either had to look for a software that would actually emulate the virtual machine, or try on another machine. Being that I was still in the lab (12:50) at the time, I decided to look into QEMU and tried running it via SSH, but it was missing a library... whatever libibumad.so is.

```
● vgrigory@f35:/home/vgrigory $ cd 453
⊗ vgrigory@f35:/home/vgrigory/453 $ ~pn-cs453/QEMU/bin/qemu-system-i386 -rtc base=
  utc -net user -net nic -m 256 -hda minix_3_1_8.vmdk
  //home/pn-cs453/QEMU/bin/qemu-system-i386: error while loading shared libraries:
  libibumad.so.3: cannot open shared object file: No such file or directory
○ vgrigory@f35:/home/vgrigory/453 $
```

I was not about to debug that.

Solutions:

Once I got home, I got onto my Windows laptop, installed VMWare Workstation, set up a Minix virtual machine, and replaced the VMDK in the directory with the provided one. Once I booted the VM, it took like 10 minutes to start, which was longer than it took to set the whole thing up.

Lessons Learned:

M series chips only want to work with ARM based virtualization. For compatibility, literally anything else works better than Apple Silicon. I wish I had more knowledge to tackle this problem and figure out how to translate these instructions from x86 to x64 ARM and make every Mac user's life easier, but alas. Also that bruteforcing something to work on an incompatible machine is mostly useless and just takes away time from doing something productive.

2. Log In

Approach:

Log into the virtual machine.

Problems Encountered:

The bogus problem at the bottom of the lab instructions briefly convinced me that I'd need a valid username and password to login to the system. But I don't have one? How do I make one?

Solutions:

I read the paragraph on logging in again, and noticed how it just said to login as root... So I typed in "root" for the password and wouldn't you know it, it just logged right in. How nice.

```
Minix Release 3 Version 1.8 (console)

10.0.0.1 login: root

To install additional packages, run 'pkgin'. First do a 'pkgin update'
to update the list of available packages, and then do a 'pkgin' to get
a list of commands. For example, 'pkgin install vim' installs the
'vim' package, and 'pkgin available' will list all available packages.

MINIX 3 supports multiple virtual terminals. Just use ALT+F1, F2, F3
and F4 to navigate among them.

For more information on how to use MINIX 3, see the wiki:
http://wiki.minix3.org.

To configure networking, run netconf(8)

# _
```

Lessons Learned:

Do not overthink instructions, or at least try them before freaking out about future problems. This is good life advice in general but sometimes it takes a two second instruction to realize how prevalent it is and how dumb such situations can get.

3. Create a User Account

Approach:

As root, create a user account for yourself and login as it instead of working with supervisor privileges.

Problems Encountered:

It seems by default, Minix only has one group, which is the operator. Based on in class materials, operators are the main controllers of the computer and can execute any commands, so I thought that setting my new user account as operator would be a bad idea. So here go my attempts at trying to make a new group... Unfortunately, "group add", "groupadd" and "newgrp" didn't do what I wanted.

```
MINIX 3 supports multiple virtual terminals. Just use ALT+F1, F2, F3
and F4 to navigate among them.
```

```
For more information on how to use MINIX 3, see the wiki:
http://wiki.minix3.org.
```

```
To configure networking, run netconf(8)
```

```
# adduser vic
Usage: adduser user group home-dir
# groups
operator
# id
uid=0(root) gid=0(operator)
# group add me
group: not found
# group
group: not found
# adduser vic me /home/vic
adduser: group me does not exist
# groupadd
groupadd: not found
# newgrp
newgrp: not found
#
```

These were after I went through Minix and Linux manpages and forums on what would create new groups. I thought that adduser would create the group listed in the command, but I guess not. The other attempts also failed because the command didn't exist in this Minix version.

I was thinking of giving up and just letting myself be the operator, but after I created my user, I had an idea.

```
# adduser vic operator /home/vic
cpdir /usr/ast /home/vic
chown -R 13:0 /home/vic
echo vic::0:0::: >>/etc/shadow
echo vic:##vic:13:0:vic:/home/vic: >>/etc/passwd
```

The new user vic has been added to the system. Note that the password, full name, and shell may be changed with the commands passwd(1), chfn(1), and chsh(1). The password is now empty, so only console logins are possible. Also note that a new operator needs an executable search path (\$PATH) that does not contain the current directory (an empty field or . in \$PATH).

```
# passwd vic
Changing the shadow password of vic
New password:
Retype password:
# chgrp
Usage: chgrp [owner:]group file...
# newgroup
newgroup: not found
# _
```

Solutions:

According to [this](#) Minix manpage, all of the credentials are stored in files in the /etc/ directory. Theoretically, as root, I could go in and mess with them... I cat'ed the group and passwd files to make sure I knew the format I needed to edit. Also why is Andrew Tanenbaum here?

```
www:*:9:
driver:*:10:
server:*:11:
sshd:*:23:
smtpd:*:26:
nogroup:*:99:
# cat passwd
root:##root:0:0:Big Brother:/root:
daemon:*:1:1:The Deuce:/etc:
bin:##root:2:0:Binaries:/home/bin:
uucp:*:5:5:UNIX to UNIX copy:/usr/spool/uucp:/usr/bin/uucico
news:*:6:6:Usenet news:/usr/spool/news:
ftp:*:7:7:Anonymous FTP:/usr/ftp:
ast:*:8:3:Andrew S. Tanenbaum:/home/ast:
www:*:9:9:World Wide Web:/usr/www:
driver:*:10:10:Device Drivers:/:
server:*:11:11:OS Servers:/:
service:*:12:12:System Services:/:
sshd:*:22:22:sshd:/:
smtpd:*:25:25:smtpd:/:
games:*:9998:98:/:
nobody:*:9999:99:/tmp:
vic:##vic:13:0:vic:/home/vic:
# tty: ignoring unrecognized escaped scancode 0x5b
# _
```

Anyway, I decided to put my user in the 'other' group, which was number 3. So I replaced the 0 in my user line in the passwd file to 3. I had to [learn](#) a bit of vim which caused some hassle but I got it.

```
smtpd:*:23:23:smtpd:/:  
games:*:9998:98::/  
nobody:*:9999:99::/tmp:  
vic:##vic:13:3:vic:/home/vic:  
~  
~
```

Now it was time to reboot and log in...

```
Minix Release 3 Version 1.8 (console)  
  
10.0.0.1 login: vic  
Password:  
  
To install additional packages, run 'pkgin'. First do a 'pkgin update'  
to update the list of available packages, and then do a 'pkgin' to get  
a list of commands. For example, 'pkgin install vim' installs the  
'vim' package, and 'pkgin available' will list all available packages.  
  
MINIX 3 supports multiple virtual terminals. Just use ALT+F1, F2, F3  
and F4 to navigate among them.  
  
For more information on how to use MINIX 3, see the wiki:  
http://wiki.minix3.org.  
  
To configure networking, run netconf(8)  
  
$ id  
uid=13(vic) gid=3(other)  
$
```

Success!

Lessons Learned:

There's always alternative, if weird, things to get around lack of commands, since commands really just build off of each other. I kind of got to this point when the network settings on my Raspberry Pi started acting up and I needed to edit files with nano to get it working again so I'm not unfamiliar with editing configuration files. I'd rather not have to do that in case I mess something up (since humans are much less reliable than computers) but at least now I know where these credentials are located and how they're read.

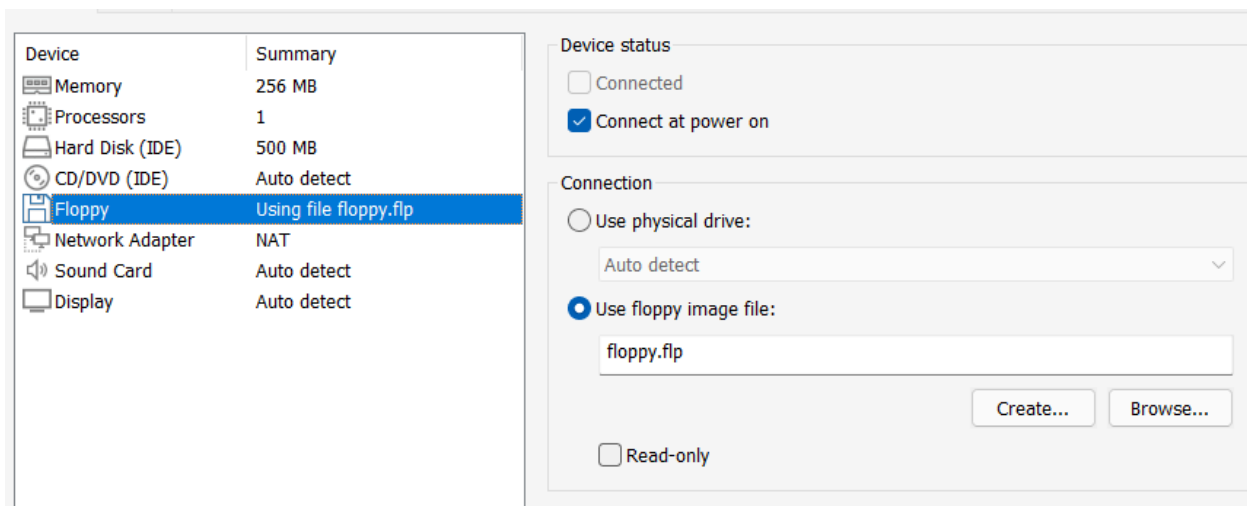
4. Create a Minix Disk Image and Use It to Store Data

Approach:

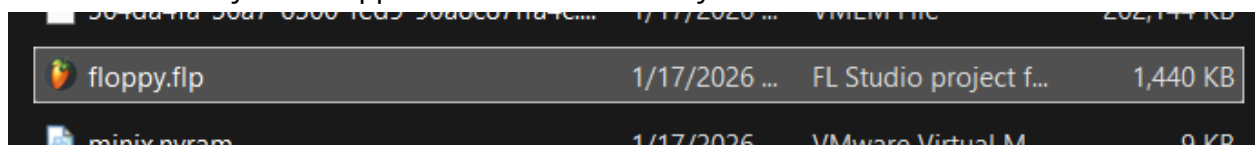
Create a virtual floppy drive to connect and mount it to the Minix system, then format it to a compatible file system.

Problems Encountered:

Since I'm using VMWare Workstation, it doesn't give me an option to specify the size of the floppy drive. I was a bit worried that this would cause issues since there are file system size limits or some systems not being able to recognize larger disk partitions, but I hoped that VMWare would be able to properly set up at least the basics and that I wouldn't need to specify too much...

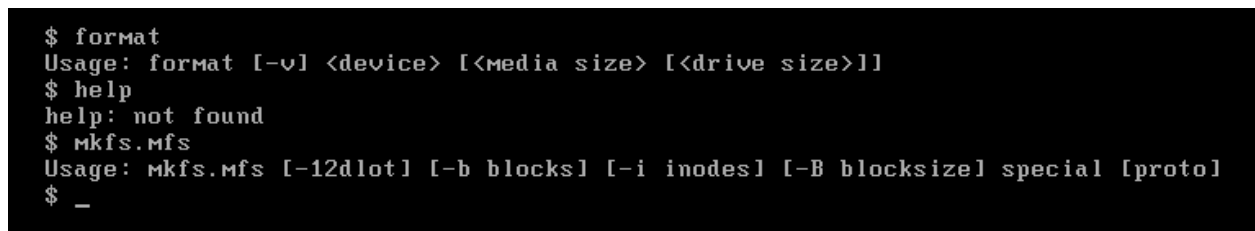


... which is exactly what happened. A nice 1440 kilobytes.



(I have FL Studio installed on this machine.)

The next issue was trying to get Minix to recognize the floppy drive. I didn't know what drive letter was attached to it or what it was referred to in Minix.



Running mount gave me things I couldn't really decipher as the output.

```
$ mount
/dev/c0d0p0s0 is read-write mounted on / (type 3)
/dev/c0d0p0s2 is read-write mounted on /usr (type 3)
/dev/c0d0p0s1 is read-write mounted on /home (type 3)
$ _
```

I guessed that the /dev/ directory might have something of note in there, but I tried running `ls /dev/` and it didn't give me much more to work with.

```
c1d7p3s3
console
fd0
fd0p0
fd0p1
fd0p2
fd0p3
fd1
fd1p0
fd1p1
fd1p2
fd1p3
filter
hello
imgrd
ip
ip0
ipstat
kbd
kbdaux
klog
kmem
log
lp
byte 2962_
```

Solutions:

This is when I decided to jump forward to section 5 and read in the dirty tricks section where it says to mount /dev/fp0. I decided to try that since it was better than having no leads for the device name. I then realized this was stated in the directions for this section but it just didn't click the first time. Amazing. Here I go mounting the thing: just mount /dev/fd0 /mnt/floppy.

```

$ mkfs fd0
$ mount fd0
Usage: mount [-r] [-t type] [-o options] special name
$ mount -t minix /fd0/ ../mnt/floppy
mount: Can't mount /fd0/ on ../mnt/floppy: Invalid argument
$ cd ..
$ ls
bin  boot  dev  etc  home  mnt  root  sbin  tmp  usr  var
$ mount -t minix /dev/fd0 /mnt/floppy
mount: Can't mount /dev/fd0 on /mnt/floppy: Invalid argument
$ cd mnt
$ ls
$ mkdir floppy
$ cd ..
$ mount -t minix /dev/fd0 /mnt/floppy
mount: Can't mount /dev/fd0 on /mnt/floppy: Invalid argument
$ mount /dev/fd0 /mnt/floppy
/dev/fd0 is read-write mounted on /mnt/floppy
$

```

Simple, kind of? It took me a while to figure out which arguments to add or not and that -t minix results in a syntax error but it works!

Lessons Learned:

Read the instructions in the lab manual. It's also useful to actually figure out what every piece of information in the instructions mean, and it's useful to read even the parts you don't plan on executing. The dirty tricks section showed me a lot of useful syntax that I was able to modify to get the result I wanted, otherwise I would've still been sitting here wondering what the device name is.

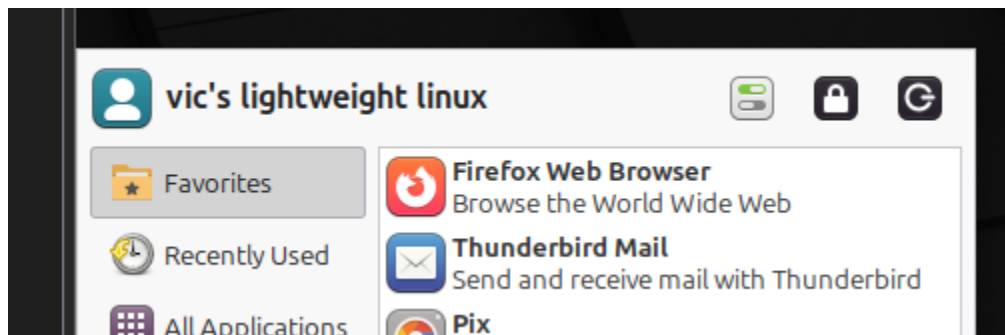
5. *Accessing Your Data from Outside Minix*

Approach:

Take one of three options listed to take data off of the floppy drive file and see it as a filesystem. The filesystem can be read directly by mounting it to the computer, using the provided files in SSH to read it after transferring the floppy drive file there, set up networking for Minix, or collect everything into a tar archive on the disk and rename it to directly extract it from another computer.

Problems Encountered:

It technically would be a problem that I don't have a Linux machine, but then I snapped back to reality and remembered that I just installed VMWare Workstation, I could just install Linux Mint... so I decided to go do that and take a break from command line operations.



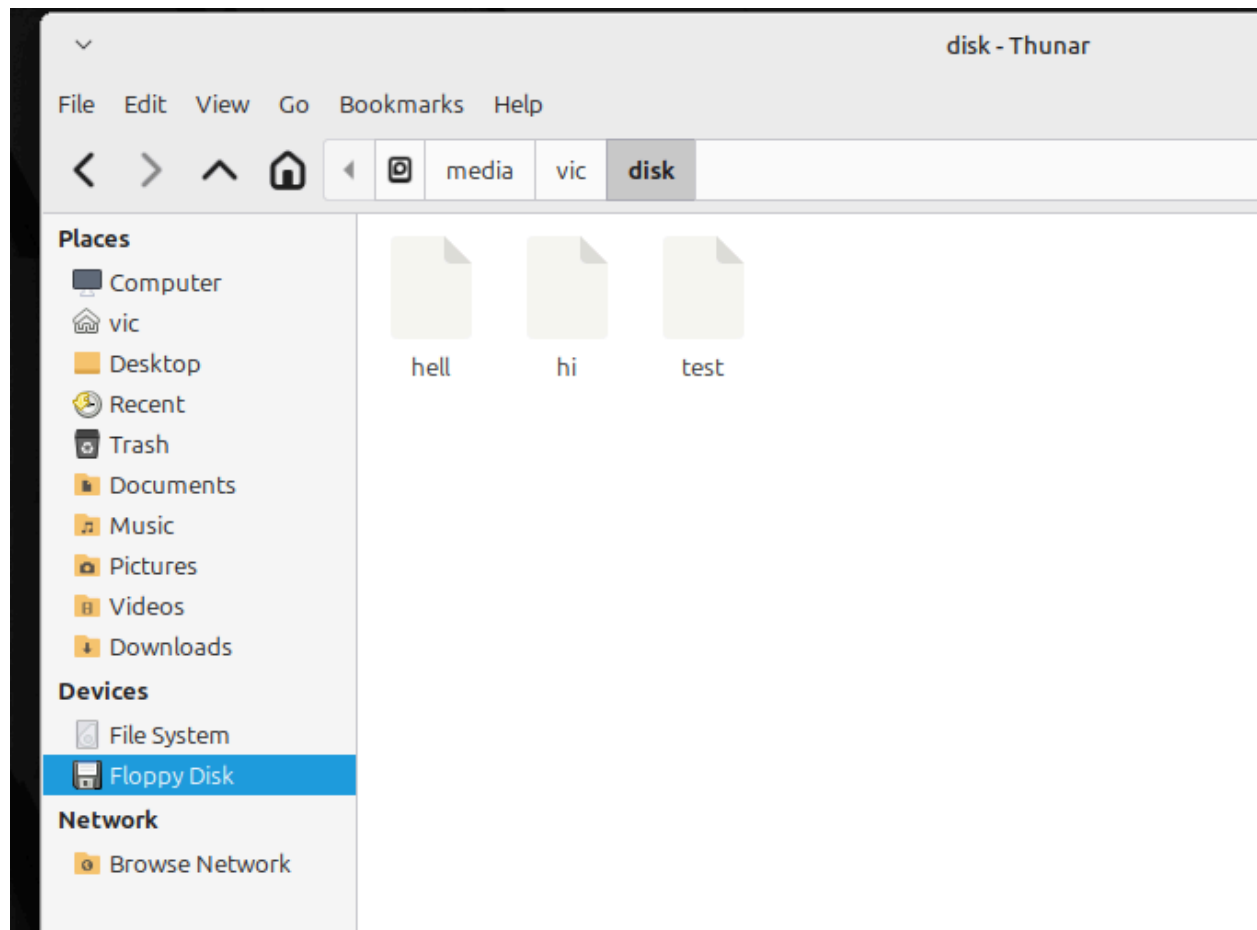
Behold, a nice, fresh installation of Linux Mint 22.3, Xfce.

Before unmounting the drive in Minix, I decided to create test files as a way to see if the floppy disk was recognized in Linux. If the drive is empty, how am I supposed to know if Linux can actually read it? After that I unmounted the drive.

```
$ mount /dev/fd0 /mnt/floppy
/dev/fd0 is read-write mounted on /mnt/floppy
$ cd /mnt/floppy
$ ls
$ touch hi
$ touch test
$ touch hell
$ ls
hell  hi  test
$ cd
```

After adding the floppy drive as a removable device in VMWare settings (the Minix VM needed this, so I thought the Linux one would too), I booted up Linux. I was a bit worried that because I installed a lightweight version of the OS, it might not come with all of the extensions needed to read the Minix filesystem. But also if the floppy drive was mounted, would that remove the need to mount it myself via the terminal?

Solutions:



Turns out Linux just natively mounted the drive and it shows up in the file explorer. Simple enough, I guess.

Lessons Learned:

Sometimes its better to let the computer do things for you - it's more reliable and less prone to messing something up. I know how the command `mount -t minix /dev/fd0 /mnt/floppy` works, which would mount the device (as it did in the Minix system), but it appears that doing so through VMWare eliminated the need to do this because Linux was able to recognize something being mounted through the VM settings, and as a bonus Linux did have the module to read Minix filesystems! I find that it's important to know the theory before you let the computer do these things for you (and that's my biggest gripe with AI).

I also do want to bring up the point with not fiddling around with the other methods when there is an easy solution. I could've spent hours trying to set up networking or archiving the device contents with `tar`, or using the provided files in the SSH to do this, but ultimately we're trying to achieve something here and there's no use making it more difficult that it needs to be if our goal is to learn to mount and unmount drives and filesystems.

6. Clean Up and Shut Down

Approach:

Use the shutdown command to properly shut down the Minix machine.

Problems Encountered:

Ideally it would just be a shutdown now command so the machine shuts down, right?

```
bin boot dev etc home mnt root s
$ umount /mnt/floppy
/dev/fd0 unmounted from /mnt/floppy
$ shutdown now
shutdown: Permission denied
$ _
```

Minix had other ideas. Permission denied for a shutdown request? Even switching to the root user didn't do anything.

```
$ login
login: root
login: /usr/adm/wtmp: Permission denied

To install additional packages, run 'pkgin'. First do a 'pkgin update'
to update the list of available packages, and then do a 'pkgin' to get
a list of commands. For example, 'pkgin install vim' installs the
'vim' package, and 'pkgin available' will list all available packages.

MINIX 3 supports multiple virtual terminals. Just use ALT+F1, F2, F3
and F4 to navigate among them.

For more information on how to use MINIX 3, see the wiki:
http://wiki.minix3.org.

To configure networking, run netconf(8)

$ shutdown now
shutdown: Permission denied
$
```

VMWare Workstation can send a Cntl Alt Delete signal to the system, but I didn't know if that's what I needed to do, or if it was the way to properly shut down the system.

Solutions:

I decided to do the Cntl Alt Delete anyway. What came up was not an instant black screen, but what resembled a shutdown sequence.

```
Broadcast message from root@10.0.0.1 (console)
Sat Jan 17 15:59:03 2026...

The system will shutdown NOW

CTRL-ALT-DEL
Local packages (down): sshd done.
Sending SIGTERM to all processes ...
MINIX will now be shut down ...
d0p0s0>
```

Sending SIGTERM to all processes seems like they're being put to sleep (duh). But after waiting for a while, the terminal was still hanging at the d0p0s0 prompt... [This](#) page said that it was safe to shut down the VM here, I wanted to make sure, so I tried to figure out what this meant.

In my search, I was led back to the Minix installation for VMWare page, where I was greeted with this it just straight up saying to shutdown the machine when this prompt came up.

When the installation is over, type

```
shutdown
```

When you get the *d0p0s0>* prompt, enter **off** to shutdown the Virtual Machine.

My next page was the QEMU Minix installation instruction page, which had some commands being given to this prompt that resembled something boot related.

```
By default, MINIX 3 will automatically load in 3 seconds.
Press ESC to enter the monitor for special configuration.
Hit a key as follows:
    1 Start MINIX 3
    2 Start Small MINIX 3 (uses less memory)
    3 Start Custom MINIX 3
[ESC]
d0p0s0>qemu_pci=1
d0p0s0>save
d0p0s0>boot
```

What this leads me to believe is that d0p0s0 is the bootloader, and that being brought back to the bootloader prompt implies that Minix is fully shutdown at that point, so I just shut it down via VMWare Workstation (in a way, clicking the dreadful X button).

Lessons Learned:

Click and type things and learn what they do. Shutting down once improperly is not going to destroy your machine, but it will teach you what a button or command does, and in this case bypasses that permission denied for shutdown. I think a similar approach to these things is important because it's the exploration and curiosity that leads one to learn about their environment, not being afraid of the consequences that a misstep can produce. I also learned that usually something critical is what really destroys machines, like deleting all of their important boot files, and not failing to flush caches, so playing around with these commands is important to becoming familiar with what we're working with.

If you got here then congratulations, you haven't died of boredom of reading through what I've been doing for the last 4 or so hours! It's dark outside.